

Resolução Normal 21/22

16 de julho de 2022 20:47

1. De que forma é possível aceder a um elemento HTML através de código JavaScript?
- a) `getElementById('idname')`.
 - b) `querySelector()`.
 - c) `getElementByClass('idname')`.
 - d) Todas as outras opções.

a)

2. Em JavaScript um Array é:
- e) *primitive type*, que permite a indexação cujo valor se inicia em 0.
 - f) *primitive type*, que permite a indexação cujo valor se inicia em 1.
 - g) *reference type*, que permite a indexação cujo valor se inicia em 0.
 - h) *reference type*, que permite a indexação cujo valor se inicia em 1.

g)

3. A *anonymous function* assume uma importância crucial no contexto do JavaScript, este tipo de função é declarada:
- a) com nome e pode ser incorporada numa expressão, possibilitando dessa forma a sua chamada a partir de qualquer ponto do script
 - b) sem nome e pode ser incorporada numa expressão, possibilitando dessa forma a sua chamada a partir da definição da expressão.
 - c) com nome e pode ser incorporada numa expressão, possibilitando dessa forma a sua chamada a partir da definição da expressão.
 - d) sem nome e pode ser incorporada numa expressão, possibilitando dessa forma a sua chamada a partir de qualquer ponto do script.

b)

4. Uma função *callback* em JavaScript
- a) é o mesmo que uma declaração de função.
 - b) é uma função que é passada para outra função como argumento ou parâmetro.
 - c) é o mesmo que uma função de expressão.
 - d) é uma função que se invoca a si própria.

b)

5. No contexto da propagação de eventos em JavaScript, a fase de *event bubbling*
- a) inicia com a captação do evento do elemento mais externo e, de seguida, propagando-o para o elemento mais interno.

5. No contexto da propagação de eventos em JavaScript, a fase de *event bubbling*
- a) inicia com a captação do evento do elemento mais externo e, de seguida, propagando-o para o elemento mais interno.
 - b) inicia com a captação do evento do elemento mais interno e, de seguida, propagando-o para o elemento mais externo.
 - c) é o momento em que a captação do evento inicia.
 - d) é o momento em que a captação do evento termina.

2)

6. Quando comparada com uma estrutura clássica (*Multi Page Application*) a implementação de *Single Page Application (SPA)*:
- a) origina um aumento do número de pedidos de dados ao servidor.
 - b) origina uma diminuição dos pedidos de dados ao servidor.

origina uma diminuição dos pedidos de dados ao servidor, e a gestão dos pedidos é feita pelo lado do cliente.

7. Qual o objetivo do *strict mode* em JavaScript?
- a) Permite uma verificação de erros menos rigorosa, facilitando a apresentação da página que contém JavaScript.
 - b) Pode ser aplicado em todo o código ou em blocos de código de forma a introduzir uma mais rigorosa verificação de erros durante a execução do JavaScript.
 - c) Permite que o JavaScript só seja executado em modo de desenvolvimento.
 - d) Permite que o JavaScript seja executado em browser mais antigos.
- (0.5v)

2)

8. De que forma é possível criar um objeto em JavaScript?
- a) Recorrendo a uma função construtora.
 - b) Através do método *Create*.
 - c) Recorrendo à sintaxe literal para criação de objetos ou recorrendo a uma classe.
 - d) É possível com todas as opções mencionadas.
- (0.5v)

2)

9. Qual o resultado de `0 === "0"` em JavaScript?
- a) `True`.

- d) É possível com todos os tipos de dados.
9. Qual o resultado de `0 === "0"` em JavaScript
- a) True.
 - b) False.
 - c) *undefined*.
 - d) null.

b)

10. Em JavaScript, o output de `console.log(1+2+'3')` é
- a) 123
 - b) 33
 - c) 6
 - d) Error

b)

11. O JavaScript é uma linguagem
- a) Compilada, sendo que o browser executa o código compilado.
 - b) Interpretada, sendo que o interpretador do browser traduz todo o código e, depois, executa-o.
 - c) Interpretada, embora os browsers mais modernos usam a tecnologia conhecida como *JIT compilation*.
 - d) Compilada, pois browsers mais modernos usam a tecnologia conhecida como *JIT compilation*. (0.5v)

c)

12. O que é o Virtual DOM no React?
- a) É a única forma de se conseguir aceder ao *Document Object Model* através de uma aplicação React.
 - b) Representa o documento HTML existente no browser, com uma estrutura de árvore lógica.
 - c) Cópia do DOM para melhorar a performance de uma aplicação React.
 - d) É a forma como o React divide o documento em diferentes componentes. (0.5v)

c)

13. Qual das afirmações se encontra correta no contexto de especificação de componentes React?
- a) A implementação de estados em React é possível e implementada da mesma forma, independentemente do tipo de componente
 - b) Os componentes de *class* e *id* do React permitem definir componentes com estado
 - c) Os hooks permitem adicionar novas funcionalidades de estados aos componentes de *class*
 - d) Só é possível implementar estados em componentes funcionais com recurso aos hooks (0.5v)

d)

14. Qual das afirmações é correta no contexto de React?
- a) O estado é mutável mas as props não
 - b) O estado é imutável mas as props não
 - c) Tanto o estado como as props são mutáveis

- Qual das afirmações é correta no contexto de React?
- a) O estado é mutável mas as props não
 - b) O estado é imutável mas as props não
 - c) Tanto o estado como as props são mutáveis
 - d) Tanto o estado como as props são imutáveis

a)

15. É possível implementar dois componentes React num único ficheiro?
- a) Sim.
 - b) Não.
 - c) Sim, apenas se forem componentes do mesmo tipo.
 - d) Apenas se os exportarmos a todos como módulos.

a)

16. Os componentes são elementos essenciais em React, sendo que
- a) todos os componentes, independente do tipo, devem incluir o método render().
 - b) apenas os componentes de class devem ter o método render().
 - c) pode conter vários métodos render().
 - d) apenas os componentes funcionais devem ter o método render().

b)

17. A renderização de um componente funcional não puro
- a) é efetuada sempre que o estado desse componente se altera, atualizando também todos os componentes filhos.
 - b) sempre que o estado existente no componente filho se altera.
 - c) Apenas uma única vez.
 - d) é efetuada sempre que o estado desse componente se altera, atualizando apenas esse componente.

a)

18. O que é um *Fragment* em React?
- a) pequeno bloco de código JSX.
 - b) pequeno bloco de código JavaScript.
 - c) sintaxe que permite agrupar um conjunto de elementos sem adicionar elementos extra ao DOM.
 - d) componente especial que permite fragmentar o código React.

c)

19. Em React, o `useState` é um hook que retorna
- a) Uma string.
 - b) Um array.
 - c) Um booleano.
 - d) Um objeto.

d)

d)

20. Os engines existentes nos browsers permitem interpretar páginas que recorram ao JSX (JavaScript Extension)?

- a) Sim, sendo uma extensão JavaScript, conseguem efetuar a devida interpretação.
- b) Não, o JSX não é reconhecido.
- c) Sim, quando se especifica o *transpiler*, como exemplo, o Babel.
- d) Apenas os browser mais recentes conseguem interpretar JSX.

c)

21. Qual o output do seguinte código (não tenha em consideração as mudanças de linha):

```
let exameId = 'ls';
function Exame() {
  this.exameId = 'LS1';
}
console.log(new Exame().exameId);
const exame1 = new Exame();
Exame.prototype.classId = 'Normal';
const exame2 = new Exame();
console.log(exame1.classId);
console.log(exame2.classId);
```

- a) LS1 Normal Normal
- b) LS1 Undefined Normal
- c) Undefined Normal LSN
- d) Undefined Undefined Normal

a)

22. Qual o output do seguinte código?

```
function xpto(x) {
  let txt = "";
  x.forEach((v, i) => {
    if (i >= 2)
      txt += v;
  });
  return txt;
}
console.log(xpto([0, 1, 1, 2, 3, 5]))
```

- a) 1235
- b) 11
- c) null
- d) 235

a)

23. Considere a seguinte função de expressão:

```
let exame = function (x) {  
  return x + x;  
}
```

Qual das seguintes opções implementa a *arrow function* com o mesmo comportamento que a função de expressão apresentada?

- a) `let exame => () = x + x;`
- b) `let exame = (x) => x + x;`
- c) `let exame (x) => x + x;`
- d) `let exame = (x) => { x + x; }`

(0.5v)

27

d) `let exame = (x) => { x + x; }`

24. Qual a afirmação correcta quando uma variável é declarada com *let*?

- a) O *scope* da variável é *block* e é *hoisted*, mas não é inicializada
- b) A variável tem *function scope* e é *hoisted*, mas não é inicializada
- c) O *scope* da variável é *block* e não é *hoisted*;
- d) A variável tem *function e global scope* e não é *hoisted*;

a)

25. Qual o output do seguinte código?

```
let nota = 10;  
if (nota === 10) {  
  let nota = 15;  
  console.log(nota);  
}  
console.log(nota);
```

- a) 10 15
- b) 15 10
- c) 10 10
- d) 15 15

15)

26. Qual o output do seguinte código?

```
function qqCoisa(...num) {  
  let y = 0;  
  for (let n of num)  
    y += n;  
  return y;  
}  
console.log(qqCoisa(1, 2, 4, 5));
```

17)

27. Qual o output do seguinte código?

```
const disciplina = "LS";  
disciplina="Linguagens Script";  
console.log(disciplina);
```

27. Qual o output do seguinte código?

```
const disciplina = "LS";  
disciplina="Linguagens Script";  
console.log(disciplina);
```

- a) LS
- b) Linguagens Script
- c) É apresentado uma mensagem de erro
- d) null

c)

28. Considere o seguinte bloco de código:

```
function Carro(modelo, cor = "Preto") {  
  this.modelo = modelo;  
  this.cor = cor;  
}  
Carro.prototype.getDetalhes = function () {  
  return `Carro ${this.modelo} de cor ${this.cor}!`;  
}  
console.log(new Carro("Toyota").getDetalhes())
```

Qual o output apresentado?

- a) Carro Toyota de cor Preto
- b) É apresentado um erro de código.
- c) Carro Toyota de cor Undefined
- d) Carro Undefined de cor Undefined

a)

29. O uso de módulos é um recurso extremamente importante na implementação de uma aplicação em JavaScript e React. Assim, considere o ficheiro JavaScript export.js com o seguinte conteúdo. (1v)

```
let f1 = (n1, n2) => n1 + n2;  
let ob1 = {  
  prop1: 'Jose Maria',  
  prop2: 'Morada do Jose'  
};  
let var1 = 1;  
export { ob1, var1 };  
export default f1;
```

De que forma deve especificar o import no ficheiro para aceder aos vários elementos?

- a) import f1, {ob1, var1} from './export.js'
- b) import {ob1, var1, f1} from './export.js'
- c) import ob1, var1, { função } from './export.js'
- d) import ob1 from './export.js'
import var1 from './export.js'
import função from './export.js'

a)

30. Considere o seguinte componente React e a invocação do componente

```
import React, { useState, useEffect } from "react";
export default function App(props) {
  const [state, setState] = useState("props.exame");
  const [estado, setEstado] = useState(1);
  useEffect(() => {
    setState("Exame");
  }, []);

  return (
    <>
      <h1>`Linguagens ${state}`</h1>
      <h1>`Script ${estado}`</h1>
    </>
  );
}

...
root.render(
  <StrictMode>
    <App exam="Linguagens Script" data="6 Julho 2022" />
  </StrictMode>
);
```

(1v)

Qual o output apresentado no browser?

- a) Linguagens Exame
Script 1
- b) Linguagens props.exame
Script 1
- c) Linguagens Linguagens Script
Script 1
- d) Linguagens
Script
- e) Nenhuma das opções apresentadas.

a)

e) Nenhuma das opções apresentadas.

(1v)

Considerando agora o seguinte `return`, para além dos botões, qual o texto apresentado no browser quando se clica no `bt1` e seguidamente no `bt2`?

```
return (
  <>
    <h1>`Linguagens ${state}`</h1>
    <h1>`Script ${estado}`</h1>
    <button onClick={() => setState("Script")}> bt1</button>
    <button onClick={() => setEstado(props.data)}> bt2</button>
  </>
);
```

- f) Linguagens Exame
Script 1
- g) Linguagens Script
Script 6 Julho de 2022
- h) Linguagens Exame
Script
- i) Nenhuma das opções apresentadas.

g)